

Humanoid Sensors and Actuators – Tutorial 6

9-Axis IMU Error-State Kalman Filter

Python Template Instructions

Course Instructors: HSA Teaching Team
hsa-lecture.ics@ei.tum.de

Summer Semester 2026

Goal

In this exercise you complete the 9-axis IMU orientation estimator in the `template/` folder. The estimator follows the Error-State Kalman Filter (ESKF) notation used in HSA Lecture 6:

$$\delta x_k = \begin{bmatrix} \delta \theta_k \\ \delta b_{g,k} \\ \delta a_{lin,k} \end{bmatrix}.$$

The nominal state is stored outside the linear Kalman state:

$$\hat{q}_k, \quad \hat{b}_{g,k}, \quad \hat{a}_{lin,k}.$$

The Kalman filter estimates only the small error state δx_k , injects it into the nominal state, and then resets the error mean.

The benchmark uses the provided data file `template/data/synthetic_imu_9axis.npz`. Your task is to fill the EKF equations and validate them with `template/fusion.py`.

Files You May Edit

- `template/imu_common.py`: complete EKF-1 and EKF-2.
- `template/imu_kalman.py`: complete EKF-3 to EKF-11.

The benchmark driver and data file are provided for validation. Keep them in place so the submitted output can be compared consistently.

1 Lecture 6 Notation and Code Mapping

The template variable names are inherited from the Python implementation. Use the following table to keep the code consistent with Lecture 6 symbols.

Lecture 6 symbol	Template variable	Meaning
$\omega_{m,k}$	<code>omega_m/gyroIn</code>	raw gyroscope measurement in rad/s
Δt	<code>obj.dt_sensor</code> or <code>obj.dt</code>	time step
$\hat{q}_k^-$	<code>obj.q_hat_minus</code>	predicted nominal orientation
$\hat{q}_k^+$	<code>obj.q_hat_plus</code>	corrected nominal orientation
$\hat{b}_{g,k}$	<code>obj.b_g_hat</code>	estimated gyro bias
$\hat{a}_{lin,k}$	<code>obj.a_lin_hat_minus/plus</code>	estimated linear acceleration
δx_k	<code>delta_x_hat</code>	posterior small error-state estimate
$\delta \theta$	<code>delta_theta_hat</code>	small orientation error
δb_g	<code>delta_b_g_hat</code>	gyro bias error

δa_{lin}	<code>delta_a_lin_hat</code>	linear acceleration error
r_a, r_m, r_k	<code>r_a, r_m, r_k</code>	accelerometer, magnetometer, and stacked residuals
H_a, H_m, H_k	<code>H_a, H_m, H_k</code>	observation matrices
P_k^-	<code>P_minus</code>	predicted error covariance
R_k	<code>R_k</code>	measurement noise covariance
S_k	<code>S_k</code>	innovation covariance
K_k	<code>K_k</code>	Kalman gain

2 Mathematical Reference

2.1 Prediction

Lecture 6 gives the gyro-based nominal prediction

$$\hat{\omega}_k = \omega_{m,k} - \hat{b}_{g,k-1}^+,$$

$$\hat{q}_k^- = \hat{q}_{k-1}^+ \otimes \text{Exp}(\hat{\omega}_k \Delta t), \quad \hat{b}_{g,k}^- = \hat{b}_{g,k-1}^+, \quad \hat{a}_{\text{lin},k}^- = c_a \hat{a}_{\text{lin},k-1}^+.$$

In the Python template, `Exp(·)` is implemented with `quaternion.from_rotation_vector`. The quaternion must remain normalized.

2.2 Predicted Sensor Directions

From the predicted orientation $\hat{q}_k^-$, predict the gravity and magnetic field directions in the sensor frame. In this template, $R(\hat{q}_k^-)$ stores the sensor axes as columns. Therefore the gravity prediction is implemented from the gravity-axis column, while the magnetic prediction uses the world magnetic reference m_w :

$$\hat{g}_s^- = s_g R(\hat{q}_k^-)_{:,i_g} g, \quad \hat{m}_s^- = \frac{R(\hat{q}_k^-) m_w}{\|R(\hat{q}_k^-) m_w\|},$$

where i_g is `ref.GravityIndex` and s_g is `ref.GravityAxisSign`.

The Python code uses these helper functions:

- `obj.rotmat2gravity(R_q_hat_minus)`
- `obj.rotmat2magnetic(R_q_hat_minus)`

Follow those helpers to keep the sign convention consistent with the template.

2.3 Residuals

Use the Lecture 6 residual form:

$$r_a = z_a - \hat{z}_a^-, \quad r_m = m_{m,k} - \hat{m}_s^-, \quad r_k = \begin{bmatrix} r_a \\ r_m \end{bmatrix}.$$

In this template:

- `z_a` is the accelerometer-derived gravity measurement.
- `z_a_hat_minus` is the predicted gravity from gyro integration and the prior orientation.

Therefore

$$r_a = z_a - \hat{z}_a^-.$$

2.4 Observation Matrix

For a vector $v = [v_x, v_y, v_z]^T$, define

$$[v]_{\times} = \begin{bmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{bmatrix}.$$

Lecture 6 writes the stacked accelerometer and magnetometer observation model as

$$H_k = \begin{bmatrix} H_a \\ H_m \end{bmatrix}, \quad H_a = [-[\hat{g}_s^-]_{\times} \quad 0 \quad I], \quad H_m = [-[\hat{m}_s^-]_{\times} \quad 0 \quad 0].$$

The template includes a first-order coupling between gyro bias error and the current orientation prediction. In code, `obj.buildHPart(v)` represents $-[v]_{\times}$. Therefore use

$$H_a = [h_g \quad -h_g \Delta t \quad I], \quad H_m = [h_m \quad -h_m \Delta t \quad 0],$$

where

$$h_g = -[\hat{g}_s^-]_{\times}, \quad h_m = -[\hat{m}_s^-]_{\times}.$$

2.5 Kalman Update

Use the standard Lecture 6 equations:

$$R_k = \begin{bmatrix} R_a & 0 \\ 0 & R_m \end{bmatrix}, \quad S_k = H_k P_k^- H_k^{\top} + R_k,$$

$$K_k = P_k^- H_k^{\top} S_k^{-1}, \quad \delta \hat{x}_k = K_k r_k.$$

In code, P_k^- is `P_minus` and R_k is `R_k`.

2.6 Error Injection

Lecture 6 states that the estimated error is injected into the nominal state:

$$\hat{q}_k^+ = \delta q(\delta \hat{\theta}) \otimes \hat{q}_k^-, \quad \hat{b}_{g,k}^+ = \hat{b}_{g,k}^- + \delta \hat{b}_g, \quad \hat{a}_{\text{lin},k}^+ = \hat{a}_{\text{lin},k}^- + \delta \hat{a}_{\text{lin}}.$$

The Python template uses one residual convention throughout the update. For this code, apply the orientation correction on the right side using $\text{Exp}(-\delta \hat{\theta})$, then normalize the quaternion. The vector parts use the corresponding subtraction convention: subtract $\delta \hat{b}_g$ from $\hat{b}_g$, and subtract $\delta \hat{a}_{\text{lin}}$ from $\hat{a}_{\text{lin}}$.

2.7 Covariance Update

Lecture 6 introduces the Joseph form

$$P_k^+ = (I - K_k H_k) P_k^- (I - K_k H_k)^{\top} + K_k R_k K_k^{\top}.$$

For this template, the expected implementation may use the simplified update

$$P_k^+ = P_k^- - K_k H_k P_k^-.$$

The later code propagates this posterior covariance into the next `P_minus`.

3 Tasks and Points

This assignment has **100 points**. Keep all array shapes explicit: 3-vectors should be shaped consistently as $(3,)$, $(1,3)$, or $(3,1)$ according to the surrounding code.

Grading style. Each task is graded holistically. Full credit means the implemented equation is mathematically correct, uses the lecture notation consistently, preserves the expected array shapes, and integrates with the provided code. Partial credit is given for submissions that follow the intended model but contain minor sign, shape, or normalization mistakes.

T.0 Submission Constraints and Validation (10 points)

Principle. A reproducible EKF exercise needs the same code boundaries and the same measurement data for every submission. The benchmark should therefore reflect the implemented equations in the TODO blocks.

Implementation. Only edit the intended TODO blocks in `imu_common.py` and `imu_kalman.py`. Preserve the public function signatures, keep the provided data file in place, and make sure the completed EKF path has no remaining `NotImplementedError`. Submit the benchmark output from:

```
cd template
python3 fusion.py
```

Grading (10 points). Full credit requires a clean submission that respects the template boundaries and runs the benchmark. Points are reduced for changing public APIs, modifying the benchmark data, leaving TODOs in the EKF path, or omitting benchmark evidence.

T.1 EKF-1: Gyro Prediction in `predictOrientation` (10 points)

Principle. The gyroscope gives short-term angular velocity. After removing the estimated bias, the angular velocity is integrated over one sample period and converted into a quaternion increment.

Implementation. In `predictOrientation`, compute

$$\hat{\omega}_k = \omega_{m,k} - \hat{b}_{g,k-1}, \quad \Delta\hat{\theta}_k = \hat{\omega}_k \Delta t.$$

Use `obj.dt_sensor` for Δt . For each increment, use the quaternion helper for the exponential map:

```
delta_q = quaternion.from_rotation_vector(delta_theta[i])
```

Then update the orientation in time order with `q_hat = q_hat * delta_q`.

Grading (10 points). Full credit requires correct bias removal, time integration, quaternion conversion, composition order, and sign normalization. Partial credit is given when the prediction is conceptually correct but has a minor broadcasting, time-step, or quaternion-order issue.

T.2 EKF-2: Predicted Gravity Direction (6 points)

Principle. The accelerometer can correct orientation because, in slow motion, it mainly measures gravity. The filter must predict what gravity should look like in the sensor frame from the current orientation.

Implementation. In `rotmat2gravity`, implement the projection directly from the rotation matrix. Select the gravity axis from `R[:, ref.GravityIndex]`, apply `ref.GravityAxisSign`, and scale by $g = 9.81 \text{ m/s}^2$. No quaternion conversion is needed in this helper.

Grading (6 points). Full credit requires the correct axis, sign, and gravity magnitude. Partial credit is given for a correct structure with a single convention mistake.

T.3 EKF-3: Quaternion to Rotation Matrix (4 points)

Principle. The residual models compare measured sensor vectors with vectors predicted from the prior orientation. Those predictions are easiest to compute using the rotation matrix $R(\hat{q}_k^-)$.

Implementation. Complete `R_q_hat_minus` with the quaternion helper:

```
R_q_hat_minus = quaternion.as_rotation_matrix(obj.q_hat_minus)
```

Grading (4 points). Full credit requires using the helper correctly and producing a valid 3×3 rotation matrix. Partial credit is limited for hand-coded or non-normalized conversions.

T.4 EKF-4: Accelerometer Residual (8 points)

Principle. The accelerometer residual says how different the measured gravity-like vector is from the gravity vector predicted by gyro integration. The linear acceleration estimate is included because external motion contaminates the accelerometer.

Implementation. Use $\hat{a}_{\text{lin},k}^- = c_a \hat{a}_{\text{lin},k-1}^+$, form `z_a` with the template's sign convention, and compute $r_a = z_a - \hat{z}_a^-$ as `z_a - z_a_hat_minus`.

Grading (8 points). Full credit requires the decayed linear acceleration, correct sign convention, and correctly shaped residual. Partial credit is given for a correct residual idea with a minor sign or shape error.

T.5 EKF-5: Magnetometer Residual (8 points)

Principle. Gravity cannot observe yaw by itself. The magnetometer adds a heading reference by comparing the measured magnetic direction with the magnetic direction predicted from the prior orientation.

Implementation. Normalize $m_{m,k}$ row-wise and reshape the predicted magnetic direction to match the residual shape:

```
m_m_norm = m_m / np.linalg.norm(m_m, axis=1, keepdims=True)
m_s_hat_minus = obj.rotmat2magnetic(R_q_hat_minus).reshape(1, 3)
```

Then compute $r_m = m_{m,k} - \hat{m}_s^-$ with a shape compatible with r_a .

Grading (8 points). Full credit requires normalization, correct magnetic prediction, and a correctly shaped residual. Partial credit is given for a correct comparison that misses normalization or shape consistency.

T.6 EKF-6: Observation Matrix H_k (16 points)

Principle. H_k linearizes how small errors in orientation, gyro bias, and linear acceleration change the predicted sensor residuals. The accelerometer observes orientation and linear acceleration; the magnetometer mainly observes orientation.

Implementation. Build

$$H_a = [h_g, -h_g \Delta t, I], \quad H_m = [h_m, -h_m \Delta t, 0],$$

where $h_g = -[\hat{g}_s^-]_\times$ and $h_m = -[\hat{m}_s^-]_\times$ are produced by calling `obj.buildHPart(z_a_hat_minus)` and `obj.buildHPart(m_s_hat_minus)`. Stack H_a and H_m into a 6×9 matrix H_k .

Grading (16 points). Full credit requires correct accelerometer and magnetometer blocks, correct bias coupling, and a final 6×9 stacked matrix. Partial credit is given for a mostly correct linearization with one block, sign, or stacking mistake.

T.7 EKF-7: Innovation Vector and Measurement Covariance (8 points)

Principle. The Kalman update uses one innovation vector and one measurement covariance. Stacking both sensors lets the filter choose one correction that balances accelerometer and magnetometer trust.

Implementation. Stack $r_k = [r_a^T, r_m^T]^T$ as a 6×1 vector. Build $R_k = \text{blockdiag}(R_a, R_m)$, using `obj.R_a` for the accelerometer block and `obj.MagnetometerNoise` times I_3 for the magnetometer block.

Grading (8 points). Full credit requires correct residual stacking and block-diagonal covariance. Partial credit is given for correct ingredients with an incorrect shape or block order.

T.8 EKF-8: Kalman Gain and Error-State Update (12 points)

Principle. The Kalman gain converts residuals into small error-state corrections. It compares uncertainty from the state prediction with uncertainty from the measurements.

Implementation. Compute

$$S_k = H_k P_k^- H_k^\top + R_k, \quad K_k = P_k^- H_k^\top S_k^{-1}, \quad \delta \hat{x}_k = K_k r_k.$$

Store the error-state estimate as `delta_x_hat`.

Grading (12 points). Full credit requires correct innovation covariance, gain, and error-state update. Partial credit is given for a correct formula sequence with a matrix-order or inverse-placement mistake.

T.9 EKF-9: Orientation Error Injection (8 points)

Principle. The ESKF does not add orientation errors directly to a quaternion. It converts the small rotation error into a correction quaternion, injects it into the nominal orientation, and then normalizes.

Implementation. Convert the bounded $\delta \hat{\theta}$ by calling the quaternion helper, then right-multiply the correction and normalize:

```
delta_q = quaternion.from_rotation_vector(-delta_theta_hat)
obj.q_hat_plus = obj.q_hat_minus * delta_q
```

Grading (8 points). Full credit requires correct conversion, injection order, and normalization. Partial credit is given for a correct small-angle correction with a sign or multiplication-order error.

T.10 EKF-10: Bias and Linear Acceleration Correction (6 points)

Principle. After the filter estimates the error state, the nominal bias and linear acceleration estimates must absorb those errors. This is the “injection” step for the vector parts of the nominal state.

Implementation. Correct $\hat{b}_{g,k}$ and $\hat{a}_{lin,k}$ by subtracting the estimated vector errors $\delta\hat{b}_g$ and $\delta\hat{a}_{lin}$, following the template residual convention.

Grading (6 points). Full credit requires both vector corrections with the correct sign and shape. Partial credit is given if one correction is correct or if the idea is right but one sign is reversed.

T.11 EKF-11: Posterior Error Covariance (4 points)

Principle. Once the residual has corrected the state, uncertainty must also decrease in the directions that were observed. This posterior covariance is then propagated to the next time step.

Implementation. Compute P_k^+ from P_k^- , K_k , and H_k . The simplified template form

$$P_k^+ = P_k^- - K_k H_k P_k^-$$

is sufficient.

Grading (4 points). Full credit requires the correct posterior covariance expression with compatible matrix dimensions. Partial credit is given for using the right matrices in an incomplete covariance update.

4 Validation Checklist

After completing all tasks, run:

```
cd template
python3 fusion.py
```

The script should:

- finish without `NotImplementedError`;
- print 9-axis fusion RMSE and dead-reckoning RMSE;
- save `orientation_comparison.png`;
- save `error_comparison.png`;
- show that 9-axis fusion has lower RMSE than dead reckoning.